

前三讲实验复习：从数据到策略与风险暴露

肖亮海

暨南大学 信息科学技术学院

课程 GitHub: github.com/styluck/quant

实验目标

- ① 统一数据和代码目录，运行前三讲的实验。
- ② 在脚本中补全关键代码。
- ③ 检查结果：是否和课件中的结果一致。

配置 Anaconda 与 Spyder

下载安装程序

下载地址: <https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/>
指定版本: Anaconda3-2024.10-1-Windows-x86_64.exe

- ① 运行安装程序, 按默认选项完成安装。
- ② 从开始菜单启动 **Spyder**。
- ③ 选择 **Projects** → **New Project** → **Existing directory**。
- ④ 选择 lab01_03_review 文件夹, 将其新建为 Spyder Project。

环境检查

```
import numpy as np # 数值计算
import pandas as pd # 表格数据处理
np.__version__, pd.__version__ # 检查版本
```

在 Spyder 中安装并检查 BaoStock

在 Anaconda Prompt 中安装

```
pip install baostock -i https://pypi.org/simple # 安装数据接口
```

在当前 Spyder Project 中新建临时脚本并运行：

```
import baostock as bs # 行情数据接口
import pandas as pd # 表格数据处理
import matplotlib.pyplot as plt # 绘图

login = bs.login() # 登录 BaoStock
assert login.error_code == "0", login.error_msg # 检查登录状态
print("BaoStock login success") # 输出登录结果
```

实验包结构

在课程主页下载实验包，包含：

路径	内容
data/	数据 CSV 文件；后期需要替换为你们自己的本地数据源
Part I 脚本	code/lab01_ma20.py
Part II 脚本	code/lab02_return_smallcap.py
Part III 脚本	code/lab03_beta_portfolio.py
下载脚本	code/lec01_download.py
output/	课程参考图；运行后将自己的结果保存在此处

Spyder 设置

将 lab01_03_review 新建为 Spyder Project，按 Part 打开相应脚本；从对应代码页补齐 TODO 后，三份脚本均可从上到下运行，且不需要修改绝对路径。

统一数据目录：你将使用哪些文件？

课程数据源：

文件	研究区间	用途
csi300_daily.csv	2015–2026	沪深 300 价格、均线策略与市场收益
sh_stock_monthly_panel.csv	2015–2025	沪市小市值策略
10 大权重日线 CSV	2018–2025	前十大权重股均值-方差、滚动 Beta 与组合

- 日线股票文件包含原始收盘价与 `backAdjustFactor`。
- 月度面板的 `next_ret` 已对应下一持有期。
- 所有股票代码统一采用 BaoStock 格式，例如 `sh.600000`。
- 后续实验中，必须使用你们自己的本地数据源，不能依赖课程提供的 CSV。

Part I: 均线择时与未来函数

实验原理：信号何时产生，收益何时发生？

20 日均线与交易信号为

$$MA_{20,t} = \frac{1}{20} \sum_{j=0}^{19} P_{t-j}, \quad S_t = \mathbb{I}(P_t > MA_{20,t}).$$

收盘价 P_t 只能在 t 日收盘后确认，因此可实现策略必须满足

$$R_{t+1}^{\text{strategy}} = S_t R_{t+1}^{\text{CSI300}}.$$

错误回测

用 S_t 乘以 R_t ：这里出现了 data snooping，且使用了无法成交的价格进行交易。

可执行回测

用 S_t 乘以 R_{t+1} ：用 `signal.shift(1)` 确保价格可以成交，规避 data snooping 风险。

- ① 读取 `csi300_daily.csv`, 检查日期排序、重复值和收盘价缺失。
- ② 计算 20 日均线并绘制价格与均线。
- ③ 同时计算错误策略与滞后一日的可执行策略。
- ④ 使用对数纵轴比较 Buy-and-Hold、错误策略和修正策略净值。
- ⑤ 用自己的话解释：未来函数如何形成？为什么只错一天会产生巨大差异？

对应脚本

将下一页代码填入 `code/lab01_ma20.py` 的 TODO, 再逐行观察信号、仓位和净值。

Part I 代码：信号、仓位与净值

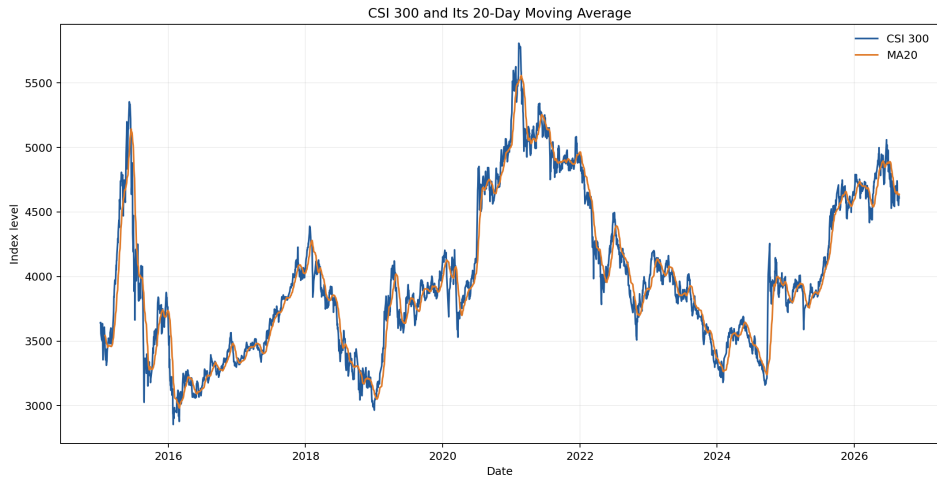
```
# TODO: 计算均线、信号、错误持仓和可执行持仓。
# 收盘价高于20日均线时，收盘后才能观察到 signal=1。
data["ma20"] = data["close"].rolling(20).mean()
data["signal"] = (data["close"] > data["ma20"]).astype(int)
data["index_ret"] = data["close"].pct_change(fill_method=None)

# 错误写法：当日收盘信号乘当日收益，使用了尚未实现的信息。
data["wrong_position"] = data["signal"]
data["wrong_ret"] = data["wrong_position"] * data["index_ret"]

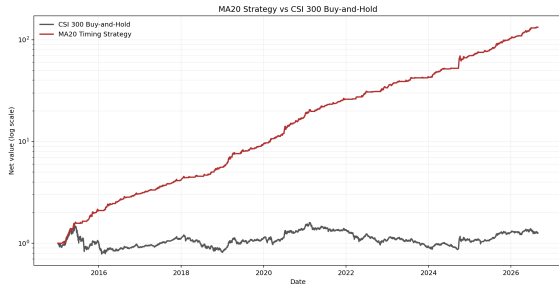
# 可执行写法：信号滞后一日，下一交易日才持仓。
data["position"] = data["signal"].shift(1).fillna(0)
data["strategy_ret"] = data["position"] * data["index_ret"]

data["index_nav"] = (1 + data["index_ret"].fillna(0)).cumprod()
data["wrong_nav"] = (1 + data["wrong_ret"].fillna(0)).cumprod()
data["strategy_nav"] = (1 + data["strategy_ret"].fillna(0)).cumprod()
```

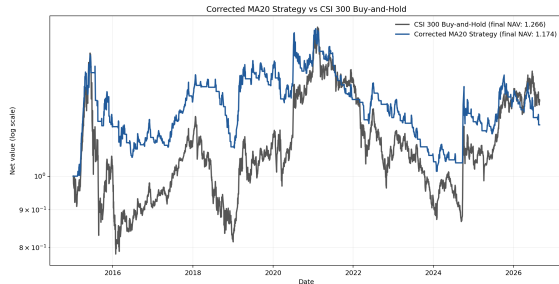
Part I 参考效果：价格与 20 日均线



Part I 参考效果：一次时间错位的影响



错误：当日信号乘以当日收益



修正：信号滞后一日

先检查信息可得时间，再讨论策略表现。

Part II: 收益统计与小市值策略

实验原理一：比较资产必须统一口径

对两只股票使用共同交易日和相同复权口径，计算

$$R_{i,t} = \frac{P_{i,t}}{P_{i,t-1}} - 1, \quad \sigma_{i,\text{ann}} = \sqrt{252} \text{Std}(R_i).$$

- 均值、波动率、Sharpe 的频率和年化系数必须一致。
- 最大回撤来自累计净值相对历史高点的跌幅。
- 两张直方图必须使用相同 bins 与横坐标范围，才能直接比较尾部。

Part II 代码：收益统计与最大回撤

```
# TODO1: 计算每只股票的回撤。  
# 先由日收益率构造净值，再相对历史最高净值计算回撤。  
nav = (1 + returns).cumprod()  
drawdown = nav / nav.cummax() - 1  
  
# TODO2: 计算年化收益、波动率和Sharpe等统计量。  
summary = pd.DataFrame({  
    # 这里使用日均收益乘252，得到算术年化平均收益率。  
    "annual_return": returns.mean() * 252,  
    "annual_vol": returns.std() * np.sqrt(252),  
    "skew": returns.skew(),  
    "excess_kurtosis": returns.kurt(),  
    "max_drawdown": drawdown.min(),  
    # 假设无风险利率为0，且收益率频率为日度。  
    "annual_sharpe": returns.mean() / returns.std() * np.sqrt(252),  
})
```

Part II 数据练习：从逐股 CSV 构造宽表

```
# TODO: 比较逐股日线宽表与月度研究面板。
# 读取 lec01_download_sse.py 下载的全部沪市逐股文件。
files = sorted(DATA_DIR.glob("sh.6*.csv"))
daily = pd.concat(
    (pd.read_csv(file, parse_dates=["date"]) for file in files),
    ignore_index=True
)

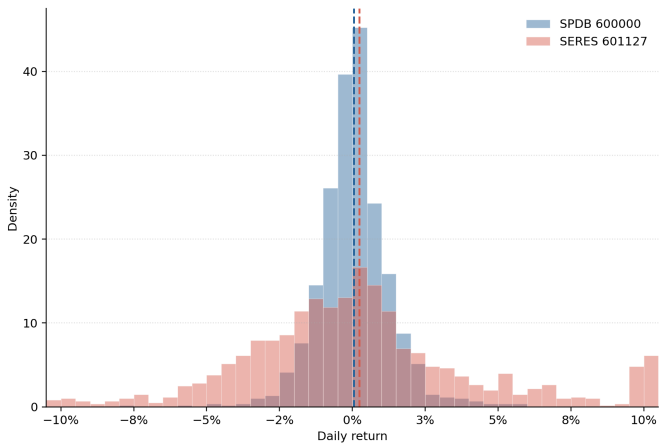
# 后复权价 = 原始收盘价 x 后复权因子。
daily["adj_close"] = (
    pd.to_numeric(daily["close"], errors="coerce")
    * pd.to_numeric(daily["backAdjustFactor"], errors="coerce")
)

close_wide = daily.pivot(index="date", columns="code", values="adj_close")
return_wide = close_wide.pct_change(fill_method=None)
```

数据边界

日线 CSV 可以构造价格与收益率宽表，但不含总市值、上市天数和下一期收益。因此小市值实验仍读取已整理的 `sh_stock_monthly_panel.csv`。

Part II 参考效果：收益率分布



Forward-adjusted close, 2021-2025; dashed lines denote means

同样的统计口径下，比较波动、偏度、峰度和尾部，而不是只比较平均收益。

实验原理二：沪市最小市值 100 只股票

每个月末在当时可用的沪市股票池中选择总市值最小的 100 只，下一月等权持有：

$$\mathcal{P}_t = \text{Bottom}100_{i \in \mathcal{U}_t^{\text{SH}}}(\text{MarketCap}_{i,t}), \quad R_{p,t+1} = \frac{1}{100} \sum_{i \in \mathcal{P}_t} R_{i,t+1}.$$

- 形成日信息只能对应下一期收益，不能使用同期收益排序后回测同期收益。
- 朴素回测先观察纸面效果；随后加入 ST、上市天数、可交易状态与最低成交额过滤。
- 高纸面收益不等于可成交、可容纳资金或扣费后可实现。

- ① 读取月度面板，仅保留 sh.6 开头的沪市股票。
- ② 按月对 market_cap 升序排名，选择前 100 只并平均 next_ret。
- ③ 构造基本过滤：非 ST、上市至少 120 天、可交易、成交额至少 1,000 万元。
- ④ 在过滤后的股票池中重新排名，不能先选 100 只再过滤。
- ⑤ 比较原始策略、过滤策略和沪深 300 的净值、波动、Sharpe 与最大回撤。

对应脚本

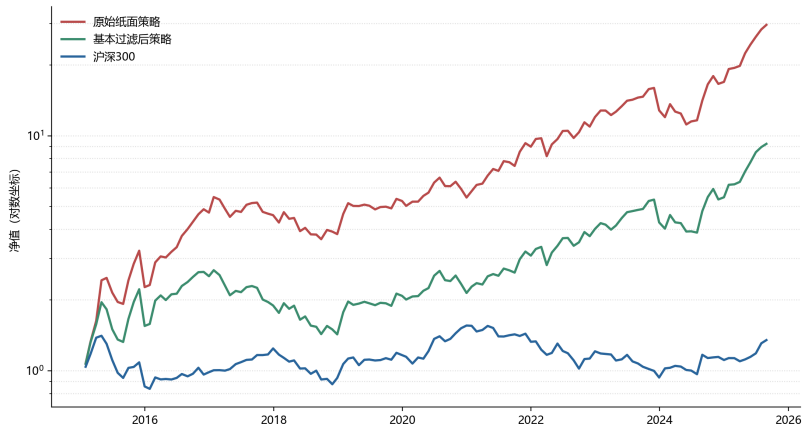
小市值排序、过滤和绩效比较位于 `code/lab02_return_smallcap.py` 后半部分。

Part II 代码：排序、持有与可实现性过滤

```
# TODO1: 按月排序并持有最小市值100只股票。
# 朴素策略：先保留市值和下一期收益都有效的样本。
naive_pool = panel.dropna(subset=["market_cap", "next_ret"]).copy()
naive_pool["size_rank"] = naive_pool.groupby("date")["market_cap"].rank(
    method="first", ascending=True
)
naive_selected = naive_pool[naive_pool["size_rank"] <= 100]
naive_ret = naive_selected.groupby("date")["next_ret"].mean()

# TODO2: 先做基本可实现性过滤，再重新排序。
# 可实现性过滤必须在市值排序之前完成。
filtered_pool = panel[
    (panel["is_st"] == 0)
    & (panel["listed_days"] >= 120)
    & (panel["tradable"] == 1)
    & (panel["amount"] >= 10_000) # 单位为千元，即至少1000万元
].dropna(subset=["market_cap", "next_ret"]).copy()
filtered_pool["size_rank"] = filtered_pool.groupby("date")["market_cap"].rank(
    method="first", ascending=True
)
filtered_selected = filtered_pool[filtered_pool["size_rank"] <= 100]
filtered_ret = filtered_selected.groupby("date")["next_ret"].mean()
```

Part II 参考效果：过滤前后的小市值策略



过滤降低了纸面收益，但尚未计入换手、滑点、冲击成本和容量约束。

Part III: 均值-方差、Beta 与组合

使用沪深 300 最新前十大权重股，研究期为 2020–2025 年；2018–2019 年数据用于滚动窗口预热。

- ① 计算十只股票的年化均值、年化标准差与样本数。
- ② 随机生成 5 万组非卖空权重，刻画均值–方差可行域。
- ③ 使用 252 个有效观察估计十只股票的滚动 Beta。
- ④ 使用静态 Beta 最小与最大的两只股票构造 $\text{Beta}=1$ 组合。

已知限制

股票池使用 2026 年权重反推历史，存在前视偏差；本实验只用于掌握计算方法。

Part III 代码：准备复权收益与共同样本

```
names = {
    "sz.300750": "宁德时代", "sz.300308": "中际旭创",
    "sh.600519": "贵州茅台", "sh.601318": "中国平安",
    "sz.300502": "新易盛", "sh.601899": "紫金矿业",
    "sh.600036": "招商银行", "sz.000333": "美的集团",
    "sh.603259": "药明康德", "sh.688256": "寒武纪",
}
codes = list(names)
# TODO1: 由后复权收盘价计算股票日收益率。
# adjusted_close 已在脚本前面的循环中逐股生成。
adjusted_close = pd.DataFrame(adjusted_close).sort_index()
stock_ret = adjusted_close.pct_change(fill_method=None).loc[:"2025-12-31"]
# TODO2: 计算沪深300日收益率，并命名为 market。
# 市场收益使用沪深300不复权收盘价计算。
market_ret = (
    index_data.sort_values("date").set_index("date")["close"]
    .pct_change(fill_method=None).rename("market").loc[:"2025-12-31"]
)
sample = stock_ret.loc["2020":"2025"]
# 组合估计只能使用十只股票与市场都有数据的共同日期。
common = pd.concat([sample, market_ret], axis=1).dropna()
```

- 单只股票统计可使用各自有效样本；组合估计必须使用共同样本。
- 不得向前填充上市前数据，也不能把缺失收益替换为零。

任务 1-2 原理：从单只股票到组合可行域

$$\hat{\mu}_i = 252 \bar{R}_i, \quad \hat{\sigma}_i = \sqrt{252} \text{Std}(R_i).$$

对满足 $\mathbf{1}^\top \mathbf{w} = 1$ 、 $w_i \geq 0$ 的组合,

$$\mu_p = \mathbf{w}^\top \hat{\boldsymbol{\mu}}, \quad \sigma_p = \sqrt{\mathbf{w}^\top \hat{\boldsymbol{\Sigma}} \mathbf{w}}.$$

- Dirichlet 随机权重天然满足非负且权重和为 1。
- 随机散点只能近似可行域，不能证明有效边界的位置。
- 组合散点云向左延伸，体现协方差带来的分散化收益。

任务 1-2 代码：统计量与随机组合

```
# TODO1: 计算单只股票的年化均值和年化标准差。
statistics = pd.DataFrame({
    # 单只股票允许使用各自的有效观察，不要求共同上市日期。
    "mean_ann": sample.mean() * 252,
    "vol_ann": sample.std() * np.sqrt(252),
    "n_obs": sample.count(),
})

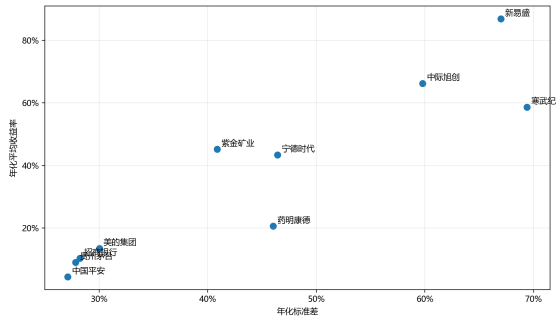
# TODO2: 估计组合使用的年化均值与协方差矩阵。
# 组合均值与协方差必须来自同一个共同样本。
stock_common = common[codes]
mu = stock_common.mean().to_numpy() * 252
sigma = stock_common.cov().to_numpy() * 252

# TODO3: 生成随机权重并计算组合均值与标准差。
rng = np.random.default_rng(20260914)
# Dirichlet权重非负且每行权重之和为1。
W = rng.dirichlet(np.ones(len(codes)), size=50000)
portfolio_mean = W @ mu
portfolio_vol = np.sqrt(np.einsum("ij,jk,ik->i", W, sigma, W))
```

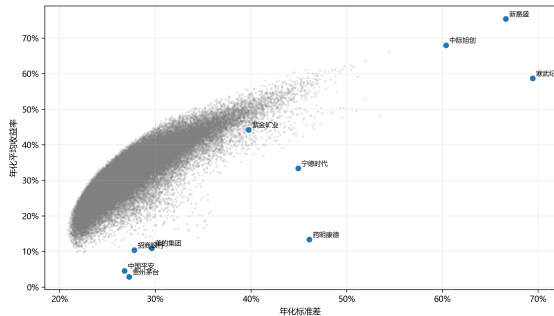
对应脚本

打开 code/lab03_beta_portfolio.py, 先运行数据准备、任务 1 和任务 2。

任务 1-2 参考效果



单只股票的均值-方差位置



5 万个非卖空随机组合

任务 3 原理：252 交易日滚动 Beta

对每只股票分别与市场收益按日期对齐：

$$\hat{\beta}_{i,t}^{(252)} = \frac{\widehat{\text{Cov}}_{t-251:t}(R_i, R_m)}{\widehat{\text{Var}}_{t-251:t}(R_m)}.$$

- 2018–2019 年用于预热；图中只展示 2020–2025 年。
- 每个窗口必须包含 252 个有效观察；不能填补上市前收益。
- Beta 是样本与模型下的估计值，不是股票永久不变的属性。

任务 3 代码：按股票计算滚动 Beta

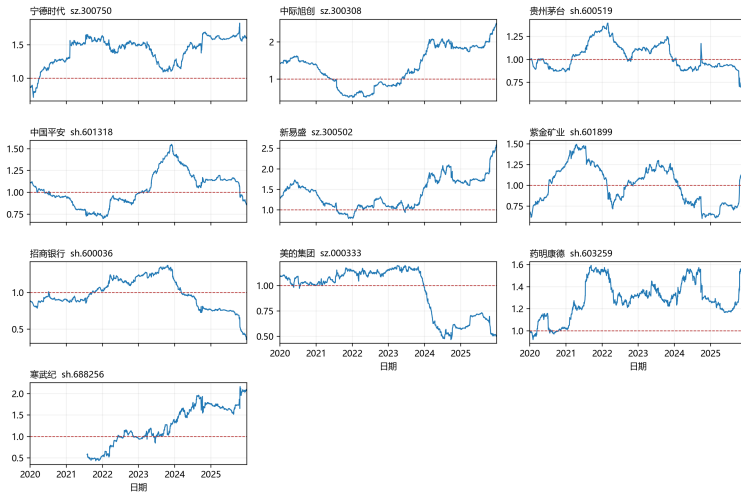
```
# TODO: 逐股计算252个有效观察的滚动Beta。
rolling_beta = {}
for code in codes:
    # 每只股票单独与市场按日期对齐，避免填充上市前收益。
    pair = pd.concat(
        [stock_ret[code], market_ret], axis=1
    ).dropna()
    # rolling(252) 要求窗口中有252个有效观察。
    rolling_beta[code] = (
        pair[code].rolling(252).cov(pair["market"])
        / pair["market"].rolling(252).var()
    )

rolling_beta = pd.DataFrame(rolling_beta).loc["2020":"2025"]
```

对应脚本

将本页代码填入任务 3 的 TODO，再检查 4×3 子图中各股票 Beta 的时变特征。

任务 3 参考效果：滚动 Beta 不是常数



任务 4 原理：用两只股票构造 $\text{Beta}=1$ 组合

选择 Beta 最小的股票 L 和 Beta 最大的股票 H ，并假设 $\hat{\beta}_L < 1 < \hat{\beta}_H$ 。令

$$w_L = \frac{\hat{\beta}_H - 1}{\hat{\beta}_H - \hat{\beta}_L}, \quad w_H = \frac{1 - \hat{\beta}_L}{\hat{\beta}_H - \hat{\beta}_L},$$

则

$$w_L + w_H = 1, \quad \beta_p = w_L \hat{\beta}_L + w_H \hat{\beta}_H = 1.$$

不要过度解释

Beta=1 只约束市场暴露，不保证组合收益、波动率或净值与沪深 300 相同。

任务 4 代码：求权重并验证约束

```
# TODO: 计算Beta=1组合的权重。
# 静态 Beta 与组合权重都使用同一个共同样本估计。
beta_values = {}
for code in codes:
    beta_values[code] = (
        common[code].cov(common["market"]) / common["market"].var()
    )
beta = pd.Series(beta_values)

low_code, high_code = beta.idxmin(), beta.idxmax()
weight_low = (beta[high_code] - 1) / (beta[high_code] - beta[low_code])
weight_high = (1 - beta[low_code]) / (beta[high_code] - beta[low_code])

weight = pd.Series(0.0, index=codes)
weight.loc[low_code] = weight_low
weight.loc[high_code] = weight_high

print("sum(weight) =", weight.sum())
print("portfolio beta =", weight @ beta)
```

对应脚本

将本页代码填入任务 4 的权重 TODO，检查权重之和与组合 Beta 是否都等于 1。

任务 4 代码：组合净值与市场比较

```
# TODO: 计算Beta=1组合净值并与沪深300比较。
# 当日组合收益是共同样本中十只股票收益率的加权和。
portfolio_ret = common[codes] @ weight

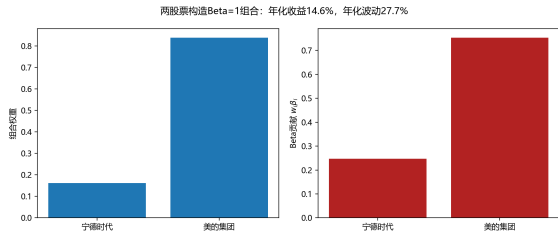
comparison_nav = pd.DataFrame({
    "Beta=1组合": (1 + portfolio_ret).cumprod(),
    "沪深300买入持有": (1 + common["market"]).cumprod(),
})

# 两条净值都从1开始，便于比较累计表现。
comparison_nav = comparison_nav / comparison_nav.iloc[0]
comparison_nav.plot(figsize=(10, 4.8), linewidth=2)
plt.ylabel("累计净值")
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

解释边界

$\beta_p = 1$ 只表示市场风险暴露相同，不表示组合净值必须复制沪深 300。

任务 4 参考效果：权重、Beta 贡献与净值



两只股票的权重与 Beta 贡献



与沪深 300 的样本净值比较

实验提交与检查清单

- ① 提交三份实的运行结果，不需要提交代码。
- ② 每个 Part 至少提交一张结果图和一段不超过 150 字的解释。
- ③ 提交实验报告。